

A FIELD PROJECT REPORT ON
STUDENT GRIEVANCE SYSTEM

Submitted in partial fulfilment of the requirements for the award of the degree

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted By

P. Lakshya	(241FA04926)
M. Sanjeev	(241FA04663)
I. Likhith	(241FA04E63)
M. Amrutha	(241FA04E76)

Under the Guidance of

Mr. Sk. Khadersha

Teaching Associate, Department of CSE



VIGNAN'S

FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH

(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

Department of Computer Science and Engineering

School of Computing and Informatics

Vignan's Foundation for Science, Technology and Research (Deemed to be University)
Vadlamudi, Guntur, Andhra Pradesh-522213, India.

April – 2026



VIGNAN'S

FOUNDATION FOR SCIENCE, TECHNOLOGY & RESEARCH

(Deemed to be University) - Estd. u/s 3 of UGC Act 1956

CERTIFICATE

This is to certify that the field project entitled "**STUDENT GRIEVANCE SYSTEM**" is being submitted by (P. Lakshya & 241FA04926), (M. Sanjeev & 241FA04663), (I. Likhith & 241FA04E63), and (M. Amrutha & 241FA04E76) in partial fulfilment of the requirements for the degree of Bachelor of Technology (B.Tech) in **Computer Science and Engineering** at Vignan's Foundation For Science Technology & Research (Deemed to be University), Vadlamudi, Guntur District, Andhra Pradesh, India.

This is a bona fide work carried out by the aforementioned students under my guidance and supervision.

S.K. Khadasha
Guide

G. Prasanthi
Project Review Committee

S.P.K.
Head of the Department
HoD
Dept. of Computer Science & Engineer
FSTR Deemed to be University
VADLAMUDI - 522 213
Guntur Dist. A.P. India

DECLARATION

Date:

We hereby declare that the work presented in the field project titled "**STUDENT GRIEVANCE SYSTEM**", is the result of our own efforts and investigation. This project is being submitted under the supervision of **(Mr. Sk. Khadarsha, Teaching Associate, CSE)**, in partial fulfilment of the requirements for the Bachelor of Technology (B.Tech) degree in Computer Science and Engineering at Vignan's Foundation for Science, Technology and Research (Deemed to be University), Vadlamudi, Guntur, Andhra Pradesh.

P. Lakshya	(241FA04926)	Signature: <u>P. Lakshya</u>
M. Sanjeev	(241FA04663)	Signature: <u>M. Sanjeev</u>
I. Likhith	(241FA04E63)	Signature: <u>I. Likhith</u>
M. Amrutha	(241FA04E76)	Signature: <u>M. Amrutha</u>

TABLE OF CONTENTS

Chapter No.		Contents	Page No
1		Introduction	1-2
	1.1	Problem Definition	2
	1.2	Existing System	2-3
	1.3	Proposed System	3
	1.4	Literature Review	4
2		System Requirements	4
	2.1	Hardware & Software Requirements	5
	2.2	Software Requirements Specification(SRS)	6
3		System Design	7
	3.1	Modules of System	7
	3.2	UML Diagrams	8
4		Implementation	9
	4.1	Sample Code	10-16
	4.2	Test Cases	17
5		Results	18
	5.1	Output Screens	18-20
6		Conclusion	21
		References	21

ABSTRACT

The “Student Grievance System” is a full-stack web application designed to streamline the process of registering, managing, and resolving student complaints within an institution.

The system allows students to submit complaints based on categories and enables other students to vote on them, helping prioritize issues effectively. Administrators can monitor all complaints through a centralized dashboard and update their status as Pending, Processing, Resolved, or Rejected.

The application is developed using HTML, CSS, and JavaScript for the frontend, and Node.js with Express.js for the backend. MongoDB Atlas is used as the cloud database for storing user and complaint data.

The system ensures efficient communication between students and administration, reduces manual effort, and improves transparency in handling issues.

1. INTRODUCTION

In modern educational institutions, effective communication between students and administration plays a crucial role in maintaining transparency, accountability, and overall institutional efficiency. One of the key areas where this interaction is essential is grievance or complaint management.

Despite the availability of digital technologies, many institutions still rely on traditional methods such as manual registers, emails, or informal communication channels for handling complaints. These approaches are often unorganized, time-consuming, and lack proper tracking mechanisms. As a result, students face difficulties in knowing the status of their complaints, and administrators struggle to manage and prioritize issues effectively.

To overcome these limitations, the “Student Grievance System” is proposed as a full-stack web-based application. This system provides a centralized platform where students can easily submit complaints, view existing issues, and track their status in real time. Additionally, the system introduces a voting mechanism, allowing students to upvote or downvote complaints, which helps in identifying and prioritizing critical issues.

The system also includes a dedicated admin dashboard that enables administrators to monitor all complaints, update their status (such as Pending, Processing, Resolved, or Rejected), and ensure timely resolution. This structured approach improves transparency, reduces duplication of complaints, and enhances communication between students and the administration.

From a technical perspective, the application is developed using HTML, CSS, and JavaScript for the frontend, Node.js and Express.js for the backend, and MongoDB Atlas for cloud-based data storage. The system follows a client-server architecture and uses REST APIs for efficient communication between components.

Overall, the Student Grievance System aims to provide a modern, scalable, and user-friendly solution for managing student grievances, thereby improving institutional efficiency and student satisfaction.

1.1 PROBLEM DEFINITION

In many educational institutions, the process of handling student complaints is still carried out through traditional and unstructured methods such as manual registers, emails, or informal communication channels. These approaches are not only inefficient but also lack proper organization and accountability.

One of the major issues with existing systems is the absence of transparency. Students often submit complaints but are unable to track their progress or know whether any action has been taken. This leads to dissatisfaction and a lack of trust in the administrative process.

Another significant problem is the lack of prioritization. All complaints are treated equally, regardless of their importance or urgency. As a result, critical issues may remain unresolved while less important ones consume administrative attention.

Additionally, the absence of a centralized system leads to duplication of complaints, increased workload for administrators, and difficulty in maintaining records. Communication between students and administrators is also inefficient, causing delays in resolution.

Therefore, there is a clear need for a structured and automated system that can provide transparency, enable complaint tracking, prioritize issues effectively, and improve overall efficiency in grievance management.

1.2 EXISTING SYSTEM

The existing systems for handling student complaints in educational institutions are either manual or partially automated web-based solutions.

Traditional systems rely on methods such as physical complaint registers, emails, or informal communication channels. These approaches are time-consuming, unorganized, and lack proper tracking mechanisms, making it difficult for both students and administrators to manage complaints effectively.

Some modern systems have been developed using web technologies such as HTML, CSS, PHP, and MySQL. These systems allow students to submit complaints online and provide basic

tracking features. Many of these systems follow a centralized architecture with role-based access control and multi-level complaint processing.

However, based on various studies and existing implementations, these systems have several limitations:

- Lack of anonymous complaint submission, which discourages students from reporting sensitive issues
- Absence of a prioritization mechanism, leading to inefficient handling of critical complaints
- Limited complaint categories, restricting proper classification of issues
- Repetition of similar complaints due to lack of visibility of existing complaints
- High administrative workload due to manual verification and processing
- Complex multi-level escalation processes, which can delay resolution

Although existing systems provide some level of automation, they fail to address key aspects such as student engagement, prioritization, and transparency.

These limitations highlight the need for an improved system that can overcome these challenges and provide a more efficient and user-friendly solution.

1.3 PROPOSED SYSTEM

The proposed system is a web-based Student Grievance System designed to provide an efficient and transparent solution for handling student grievances.

The system offers a centralized platform where students can submit complaints, view existing issues, and track their status. Complaints are categorized based on type, which helps in better organization and management.

A key feature of the system is the voting mechanism (upvote/downvote), which allows students to support existing complaints. This helps in prioritizing important issues and reduces duplication of complaints.

The system uses role-based access control, where students can submit and view complaints, while administrators manage them through a dedicated dashboard. Admins can update complaint status such as Pending, Processing, Resolved, or Rejected.

From a technical perspective, the system is developed using HTML, CSS, and JavaScript for the frontend, Node.js and Express.js for the backend, and MongoDB Atlas for data storage. The application follows a client-server architecture and communicates through REST APIs.

Overall, the proposed system improves transparency, prioritization, and efficiency in complaint management.

1.3 LITERATURE REVIEW

A study of existing student grievance systems shows that most platforms provide basic complaint submission and tracking features.

Some systems use centralized architectures with role-based access control and multi-level complaint handling. These systems improve organization and monitoring of complaints.

Advantages:

- Centralized complaint management
- Structured workflow
- Digital tracking of complaints

Limitations:

- No complaint prioritization
- Lack of anonymity
- Repetition of similar complaints
- Limited student interaction

From the study, it is clear that existing systems lack important features such as prioritization and user engagement.

The proposed system addresses these issues by introducing a voting mechanism and category-based complaint management, providing a more efficient and user-friendly solution.

2. SYSTEM REQUIREMENTS

System requirements define the necessary hardware and software components required for the successful development and execution of the Student Grievance System. These requirements ensure that the system operates efficiently, reliably, and securely.

The system is designed as a web-based application, which means users can access it through a browser without the need for additional software installation. It should support multiple users simultaneously and provide smooth performance under normal usage conditions.

The requirements are categorized into hardware requirements and software requirements, which are described in the following sections.

2.1 HARDWARE & SOFTWARE REQUIREMENTS

Hardware Requirements:

Client-Side:

- Computer/Laptop/Smartphone
- Minimum 2 GB RAM (4 GB recommended)
- Stable internet connection

Server-Side:

- **Processor:** Dual-Core or above
- **RAM:** Minimum 4 GB
- **Storage:** 50 GB or more
- Reliable network connectivity

Software Requirements:

Client-Side:

- Web Browser (Google Chrome, Microsoft Edge, etc.)
- Operating System: Windows, Linux, Android, or iOS

Development Technologies:

- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Node.js, Express.js
- **Database:** MongoDB Atlas

Additional Tools:

- Version Control: GitHub
- Deployment Platform: Render

2.2 SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

A Software Requirements Specification (SRS) defines the functional and non-functional requirements of the system. It acts as a blueprint for designing and developing the application.

Functional Requirements:

- User Registration and Login
- Complaint Submission
- Viewing and Tracking Complaints
- Voting System (Upvote/Downvote)
- Admin Dashboard for complaint management
- Status updates (Pending, Processing, Resolved, Rejected)

Non-Functional Requirements:

- Security: Secure user authentication and data protection
- Performance: Fast response time and smooth operation
- Usability: Simple and user-friendly interface
- Scalability: Ability to handle multiple users
- Reliability: System should work without failure

Assumptions:

- Users have access to the internet
- Users have basic knowledge of web applications
- System is accessed through modern web browsers

3. SYSTEM DESIGN

System Design defines the overall architecture, components, and workflow of the Student Grievance System. It acts as a blueprint that describes how different parts of the system interact with each other to perform required operations.

The system follows a three-tier architecture consisting of the Presentation Layer (Frontend), Application Layer (Backend), and Data Layer (Database). This layered approach improves system organization, scalability, and maintainability.

The frontend provides the user interface for interaction, the backend handles application logic and processing, and the database stores all necessary data such as user details and complaints.

Overall, the system design ensures efficient communication between components and smooth functioning of the application.

3.1 MODULES OF THE SYSTEM

The Student Grievance System is divided into several modules, each responsible for a specific function:

User Module:

Handles user registration, login, and authentication. It allows students to access the system securely.

Complaint Module:

Allows users to submit complaints, view existing complaints, and track their status.

Voting Module:

Enables users to upvote or downvote complaints, helping in prioritizing important issues.

Admin Module:

Provides administrators with a dashboard to view all complaints, update their status, and manage the system.

History Module:

Maintains records of previously submitted complaints and their outcomes.

These modules work together to ensure efficient and organized complaint management.

3.2 SYSTEM ARCHITECTURE

The system is designed using a three-tier architecture:

Presentation Layer (Frontend):

Developed using HTML, CSS, and JavaScript, this layer provides the user interface through which users interact with the system. It includes pages such as login, dashboard, complaint submission, and admin panel.

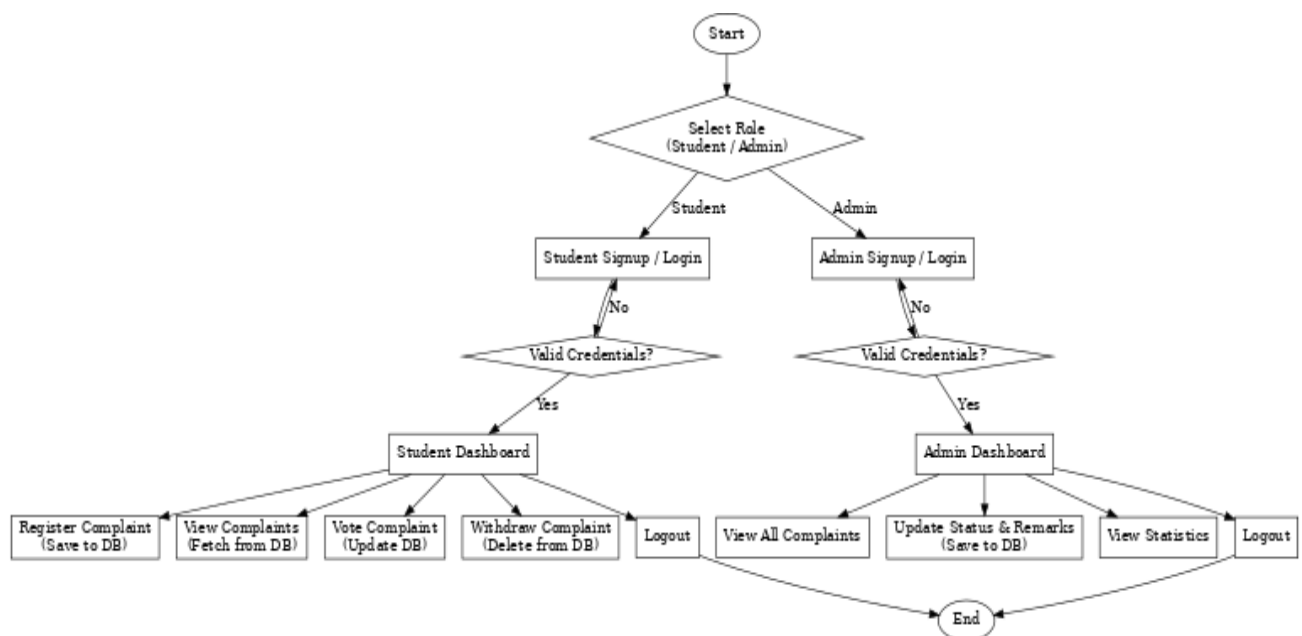
Application Layer (Backend):

Implemented using Node.js and Express.js, this layer handles business logic, API requests, authentication, and data processing. It acts as a bridge between the frontend and the database.

Data Layer (Database):

MongoDB Atlas is used as a cloud-based database to store user information, complaints, voting data, and status updates. It ensures efficient data storage and retrieval.

The frontend communicates with the backend through REST APIs, and the backend interacts with the database to perform required operations. This architecture ensures scalability, flexibility, and efficient system performance.



4. IMPLEMENTATION

The implementation phase involves developing the Student Grievance System by integrating the frontend, backend, and database components into a complete working application.

The system is developed as a full-stack web application using modern web technologies. The frontend is designed using HTML, CSS, and JavaScript to provide a simple and interactive user interface. It includes pages such as login, signup, dashboard, complaint submission form, and admin panel.

The backend is developed using Node.js and Express.js. It is responsible for handling API requests, processing user data, managing authentication, and controlling system operations. REST APIs are created for functionalities such as login, signup, complaint submission, fetching complaints, and voting.

The database is implemented using MongoDB Atlas, which stores user information and complaint data in a structured format. Each complaint contains fields such as category, description, user ID, status, votes, remarks, and timestamp.

The frontend communicates with the backend using API calls (fetch requests). The backend processes these requests and interacts with the database to perform operations such as inserting, updating, and retrieving data.

The system is deployed on Render, which hosts the backend server and makes the application accessible online. MongoDB Atlas provides cloud-based storage, ensuring data availability and scalability.

Overall, the implementation integrates all components effectively to provide a functional and efficient complaint management system.

4.1 SAMPLE CODE

Example API Route (Node.js):

```
app.post("/api/login", async (req, res) => {  
  const user = await User.findOne({ username: req.body.username });  
  if (user && user.password === req.body.password) {  
    res.send("Login successful");  
  } else {  
    res.send("Invalid credentials");  
  }  
});
```

SOURCE CODE

Login.html

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>VFSTR Complaint Portal</title>  
</head>  
<body data-page="login">  
  
  <main class="auth-shell" id="mainShell">  
  
    <section class="brand-panel">  
      <div class="resolve-btn-container">  
        <button class="resolve-btn" onclick="splitScreen()">RESOLVE</button>  
      </div>  
  
      <div class="brand-content">
```

```

    <p class="eyebrow">Campus Support Portal</p>
    <h1>VFSTR Complaint<br>Management System</h1>
  </div>

  <div class="circle-deco"></div>
</section>

<section class="form-panel">
  <form class="auth-card" id="loginForm" onsubmit="handleLogin(event)">
    <h2>Login</h2>
    <div id="errorMessage" style="color: red; font-size: 0.9rem; margin-bottom: 15px;
display: none;"></div>

    <label>Username</label>
    <input type="text" id="username" placeholder="Enter username" required>

    <label>Password</label>
    <input type="password" id="password" placeholder="Enter password" required>

    <button type="submit" class="btn-login">Login</button>

    <p style="font-size: 0.85rem; color: #888; text-align: center; margin-top: 20px;">
      New here? <a href="/frontend/signup.html" style="color: #5a7a2a; font-weight:
bold; text-decoration: none;">Create an account</a>
    </p>
  </form>
</section>

</main>

```



```

<script>

// Trigger split screen animation on page load
window.addEventListener('load', () => {
  setTimeout(splitScreen, 300);
});

function splitScreen() {
  // Adds the 'is-split' class to trigger the CSS transition
  document.getElementById('mainShell').classList.add('is-split');
}

const API_BASE =
  window.location.hostname === 'localhost' || window.location.hostname ===
'127.0.0.1'
  ? 'http://localhost:5000/api'
  : `${window.location.origin}/api`;

async function handleLogin(event) {
  event.preventDefault();

  const username = document.getElementById('username').value.trim();
  const password = document.getElementById('password').value.trim();
  const errorDiv = document.getElementById('errorMessage');

  // Clear previous error
  errorDiv.style.display = 'none';
  errorDiv.textContent = '';

  if (!username || !password) {
    errorDiv.textContent = 'Username and password are required.';
  }
}

```

```

        errorDiv.style.display = 'block';
        return;
    }

    try {
        const response = await fetch(`${API_BASE}/login`, {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
            },
            body: JSON.stringify({ username, password })
        });

        const data = await response.json().catch(() => ({ message: 'Server returned an
invalid response.' }));

        if (response.ok) {
            // Store user info in localStorage
            localStorage.setItem('scmsUser', JSON.stringify(data.user));

            // Redirect based on role
            if (data.user.role === 'admin') {
                window.location.href = '/frontend/admin-dashboard.html';
            } else {
                window.location.href = '/frontend/dashboard.html';
            }
        } else {
            errorDiv.textContent = data.message || 'Login failed. Please try again.';
            errorDiv.style.display = 'block';
        }
    }

```

```

    } catch (error) {
        console.error('Login error:', error);
        errorDiv.textContent = 'Connection error. Please check backend server and MongoDB are running.';
        errorDiv.style.display = 'block';
    }
}
</script>
</body>
</html>

```

Server.js

```

const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const path = require("path");
const bcrypt = require("bcrypt");

const authRoutes = require("./routes/auth");
const complaintRoutes = require("./routes/complaints");
const User = require("./models/User");

const app = express();
const PORT = process.env.PORT || 5000;
const MONGODB_URI =
    process.env.MONGODB_URI || "mongodb://127.0.0.1:27017/student_complaint_system";

app.use(cors({
    origin: ["https://student-complaint-system-49er.onrender.com", "http://localhost:5000", "http://localhost:3000"],
    methods: ["GET", "POST", "PUT", "DELETE"],

```

```
    allowedHeaders: ["Content-Type"],
    credentials: true
  }));
app.use(express.json());

app.use("/api", authRoutes);
app.use("/api/complaints", complaintRoutes);

const projectRoot = path.join(__dirname, "..");
app.use(express.static(projectRoot));

app.get("/", (req, res) => {
  res.redirect("/frontend/login.html");
});

async function ensureDefaultAdmin() {
  const adminUsername = "admin@123";
  const adminPassword = "admin123";
  const hashedPassword = await bcrypt.hash(adminPassword, 10);

  await User.findOneAndUpdate(
    { username: adminUsername },
    {
      username: adminUsername,
      password: hashedPassword,
      role: "admin"
    },
    {
      upsert: true,
      new: true,
```

```

    setDefaultsOnInsert: true
  }
);
}

mongoose

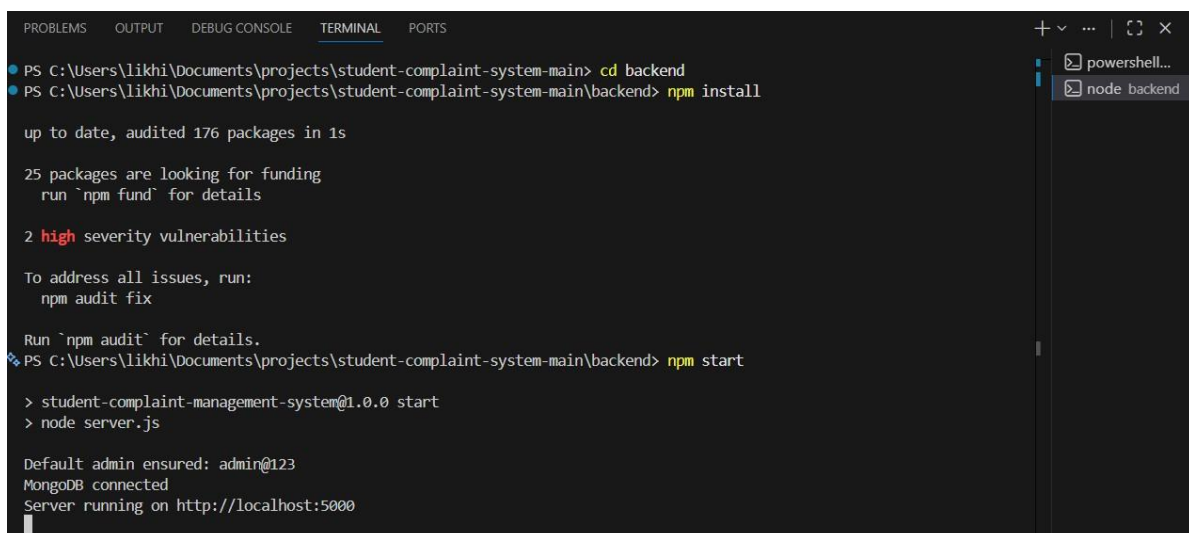
.connect(MONGODB_URI)

.then(async () => {
  await ensureDefaultAdmin();
  console.log("Default admin ensured: admin@123");
  console.log("MongoDB connected");
  app.listen(PORT, () => {
    console.log(`Server running on http://localhost:${PORT}`);
  });
})

.catch((error) => {
  console.error("MongoDB connection error:", error.message);
});

```

COMMANDS



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
• PS C:\Users\likhi\Documents\projects\student-complaint-system-main> cd backend
• PS C:\Users\likhi\Documents\projects\student-complaint-system-main\backend> npm install

up to date, audited 176 packages in 1s

25 packages are looking for funding
  run `npm fund` for details

2 high severity vulnerabilities

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
PS C:\Users\likhi\Documents\projects\student-complaint-system-main\backend> npm start

> student-complaint-management-system@1.0.0 start
> node server.js

Default admin ensured: admin@123
MongoDB connected
Server running on http://localhost:5000

```

4.2 TEST CASES

Test Case 1:

Input: Valid login credentials

Expected Output: Login successful

Test Case 2:

Input: Invalid login credentials

Expected Output: Error message

Test Case 3:

Input: Submit complaint

Expected Output: Complaint stored in database

Test Case 4:

Input: Vote on complaint

Expected Output: Vote count updated

Test Case 5:

Input: Admin updates complaint status

Expected Output: Status updated successfully

5. RESULTS

The Student Grievance System was successfully developed and tested under normal usage conditions. The system was able to perform all core functionalities efficiently, including user authentication, complaint submission, voting, and administrative control.

During testing, users were able to register and log in without errors. Students could submit complaints, view existing complaints, and participate in the voting system. The voting mechanism worked correctly, helping in identifying and prioritizing important issues.

The admin dashboard functioned effectively by allowing administrators to view all complaints, update their status (Pending, Processing, Resolved, or Rejected), and manage the overall system. Complaint status updates were reflected correctly for users, ensuring transparency.

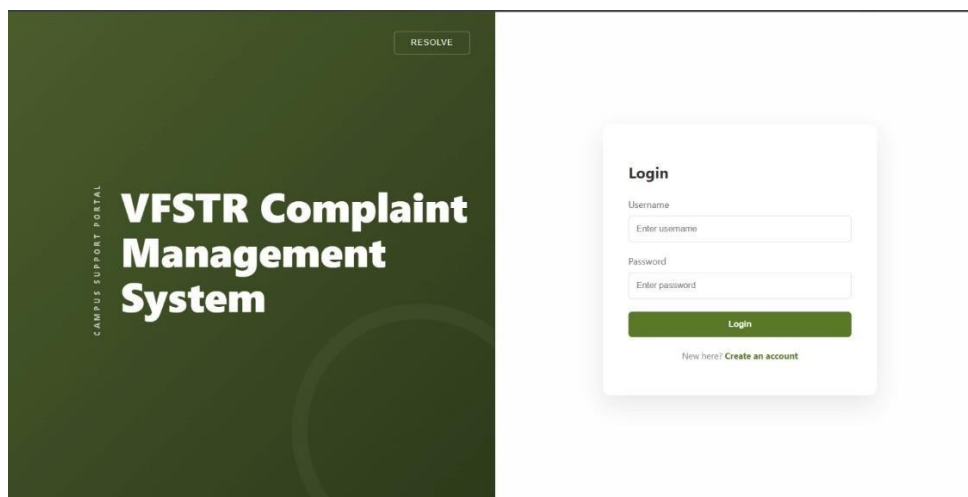
The system demonstrated smooth performance, quick response time, and proper data handling using MongoDB Atlas. The integration between frontend, backend, and database was successful, ensuring seamless communication through APIs.

Overall, the results confirm that the system meets its objectives by providing an efficient, transparent, and user-friendly solution for complaint management.

5.1 OUTPUT SCREENS

The system includes several user interface screens that demonstrate its functionality:

- **Login Page** – Allow s users to enter credentials and access the system



- **Signup Page** – Enables new users to register

Join VFSTR Complaint Portal

CREATE ACCOUNT

JOIN NOW

Create your account to submit complaints, track progress, and engage with peer issues. Choose your role - Student or Admin - and start making a difference today.

Create Account

Username
Choose username

Password
Create password (min 6 chars)

Role
Select your role

Create Account

[Already have an account? Login here](#)

- **Dashboard** – Displays complaints and system features

STUDENT PANEL

SCMS

WELCOME BACK

Welcome, Thor

Manage your complaints, check updates, and engage with issues raised by peers.

Total Complaints: **2**

My Pending: **0**

Resolved: **2**

Recent Complaints

View full history

Please change the hostel afternoon menu.

Status: Resolved | Votes: 0

Remarks: No remarks yet.

ADMIN PANEL

SCMS

OVERVIEW

Welcome, admin@123

Track complaint volume and resolution progress from one central panel.

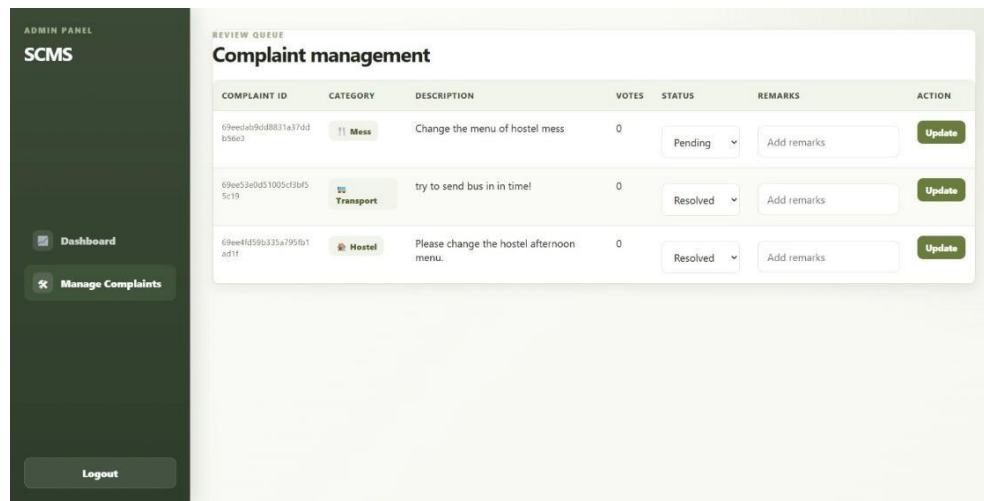
Total Complaints: **3**

Pending: **1**

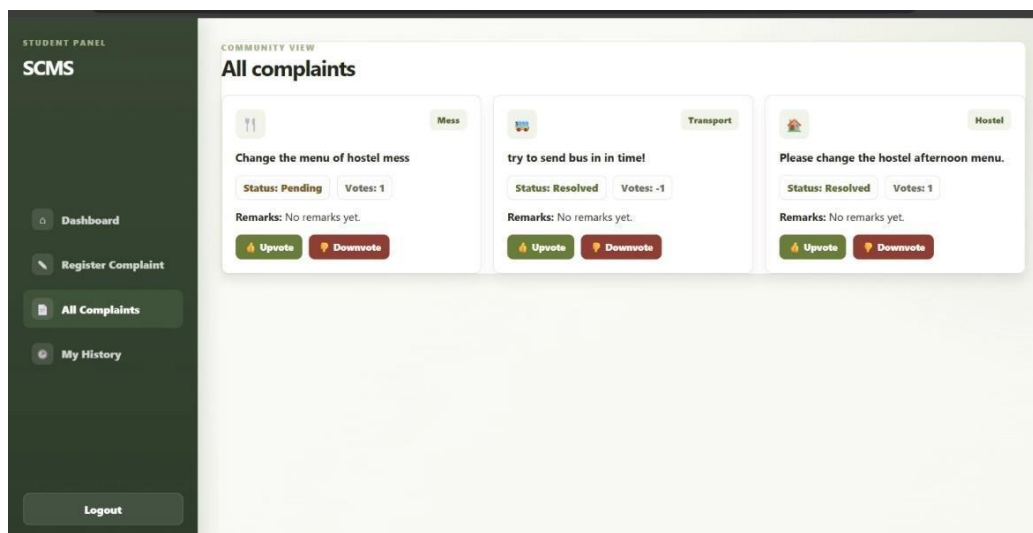
Processing: **0**

Resolved: **2**

- **Complaint Submission Page** – Allows users to submit new complaints



- **Voting Interface** – Enables upvote/downvote functionality



- **Admin Panel** – Allows administrators to manage complaints and update status



6. CONCLUSION

The Student Grievance System has been successfully designed and implemented as a full-stack web application to improve the process of handling student grievances. The system provides a centralized and user-friendly platform where students can submit complaints, track their status, and participate in prioritizing issues through a voting mechanism.

The application effectively addresses the limitations of traditional and existing systems by introducing features such as category-based complaint registration, voting, role-based access control, and an admin dashboard for efficient management. These features improve transparency, reduce duplication of complaints, and enhance communication between students and administrators.

The system demonstrates reliable performance, smooth user interaction, and efficient data handling using modern web technologies such as Node.js, Express.js, and MongoDB Atlas. Deployment on a cloud platform ensures accessibility and scalability.

Overall, the project achieves its objective of providing an efficient, transparent, and scalable solution for complaint management. It can be further enhanced with advanced features such as secure authentication, real-time updates, and mobile compatibility for broader usage.

GITHUB LINK: <https://github.com/likhithinavolu28/Student-Grievance-System>

REFERENCES

- [1] Satheeswaran Venkatesan, Arjun R, Nidhin A, and Pranav C., “State-Level Students Grievance Support System,” developed using HTML, CSS, PHP, and SQL, focusing on centralized complaint handling and role-based access control.

- [2] Moaz Hamed, Osamah Al-Harbi, Abdallah Al-mukhalf, and Hatem M. El-Boghdadi, “Design of Students Complaints Management System,” implemented using HTML, CSS, PHP, MySQL, JavaScript, and UML, focusing on automated complaint processing and tracking.

- [3] A. S. Ghotekar, S. R. Rokade, and S. S. Kadam, “Online Grievance Redressal System for Educational Institutions,” International Journal of Computer Science and Mobile Computing (IJCSMC), Vol. 9, Issue 5, 2020. This research focuses on automating the resolution of student concerns through a tiered administrative structure, utilizing HTML, CSS, JavaScript, and a backend database to replace manual paper-based filing with a real-time digital tracking mechanism.